

Universidad Autónoma de Zacatecas

Ingeniería en Software

Seminario de Investigación

SRS y SDS/CDS del prototipo

**EMPS-Fresnillo: Estimador Municipal de Proyectos de Software con viabilidad
técnico-fiscal y control de cambios**

Rogelio Flores Rodríguez

Fecha: 30 de abril de 2026

Control del documento

Nombre del sistema: EMPS-Fresnillo.

Descripción corta: Sistema de apoyo para estimar tempranamente proyectos de software municipales, comparando desarrollo tradicional, desarrollo asistido por herramientas generativas y bytecoding, e integrando costos fiscales, laborales, flujo de efectivo, riesgos de cambios y mantenimiento.

Versión: 1.0, avance de investigación.

Tipo de documento: SRS, SDS/CDS y especificación funcional para implementación en VS Code con asistencia de Claude.

Alcance del documento: Define qué debe hacer el prototipo, cómo se organizará, qué datos capturará, cómo calculará escenarios y qué reportes generará. No sustituye asesoría legal, fiscal ni contable; modela variables para análisis preliminar.

1. Visión del sistema

EMPS-Fresnillo busca resolver un problema práctico: estimar proyectos de software municipales antes de contratar o comprometer alcance. El sistema no se limita a horas de programación. Integra cuatro dimensiones:

1. Dimensión técnica: módulos, requerimientos, datos, integraciones, seguridad, pruebas, documentación y despliegue.
2. Dimensión de modo de desarrollo: tradicional, asistido por herramientas generativas, bytecoding o híbrido.
3. Dimensión de cambios y mantenimiento: bitácora de cambios, impacto, garantía, mejora, nuevo alcance y soporte posterior.
4. Dimensión fiscal-laboral-financiera: nómina, prestaciones, seguridad social, impuesto sobre nóminas, IVA, ISR, gastos, margen, calendario de cobro y capital de trabajo.

La visión es que el Ayuntamiento y el proveedor puedan ver escenarios antes de tomar una decisión. Un precio bajo no se debe aceptar sin revisar si cubre equipo, impuestos, cambios y mantenimiento. Un proveedor no debe comprometerse a un proyecto si el flujo de efectivo lo puede dejar sin capacidad de pagar al equipo.

2. Objetivos del sistema

Objetivo general

Construir un prototipo que permita estimar tempranamente proyectos de software municipales mediante escenarios técnicos, de desarrollo, fiscales, laborales y de flujo de efectivo.

Objetivos específicos

1. Capturar información estructurada del proyecto, requerimientos, módulos y contexto municipal.
2. Calcular rangos de esfuerzo técnico: optimista, probable y conservador.
3. Comparar escenarios por modo de desarrollo: tradicional, asistido, bytecoding e híbrido.
4. Registrar cambios y estimar su impacto en tiempo, costo, pruebas, capacitación y mantenimiento.
5. Simular costo de equipo con salario, prestaciones, seguridad social, impuesto sobre nóminas y otros costos patronales.
6. Calcular IVA trasladado, IVA acreditable estimado, subtotal, total, margen y resultado antes y después de impuestos.
7. Evaluar flujo de efectivo con anticipos, pagos por entregables y pago final.
8. Generar reportes separados para Ayuntamiento, proveedor y estudio académico.
9. Guardar histórico para validar la hipótesis con casos futuros.

3. Alcance

Incluido

- Captura de proyecto y área solicitante.
- Captura de módulos, historias de usuario, integraciones, datos y reportes.
- Evaluación de claridad de requerimientos.
- Comparación de modos de desarrollo.
- Estimación de esfuerzo por rango.
- Registro de riesgos y cambios.
- Cálculo de costo de equipo.
- Parámetros fiscales y laborales 2026 configurables.
- Cálculo de mantenimiento mensual y garantía.
- Reporte de decisión para Ayuntamiento.
- Reporte de viabilidad para proveedor.
- Exportación a PDF y Excel o CSV.
- Bitácora para análisis de investigación.

No incluido en la primera versión

- Nómina oficial timbrada.
- Declaraciones fiscales reales.
- Integración directa con SAT, IMSS o bancos.
- Firma electrónica.
- Gestión completa de contratos públicos.
- Sustitución de asesoría contable, fiscal o legal.
- Modelo predictivo entrenado con datos locales, hasta que exista histórico suficiente.

4. Stakeholders

Ayuntamiento. Necesita saber si la propuesta es viable, si el alcance está claro, si el mantenimiento está incluido y si el proveedor tiene una estructura sostenible.

Proveedor de software. Necesita saber si el precio cubre equipo, impuestos, herramientas, riesgos, soporte y margen.

Área usuaria. Necesita explicar procesos, datos, responsables, reportes y validaciones.

Área administrativa o financiera. Necesita revisar presupuesto, calendario de pagos, anticipos, contratos y evidencia.

Responsable técnico. Necesita estimar esfuerzo, arquitectura, seguridad, pruebas, datos y mantenimiento.

Investigador. Necesita guardar variables para validar la hipótesis y comparar escenarios.

5. Supuestos y restricciones

1. Los cálculos fiscales son preliminares y parametrizables.
2. La primera versión usará reglas, rangos y ponderaciones; no un modelo predictivo cerrado.
3. Los usuarios deberán poder modificar parámetros anuales como UMA, salario mínimo, tasas y factores de carga.
4. El sistema debe ser entendible para perfiles técnicos y administrativos.
5. El diseño debe ser compatible con implementación web local usando VS Code.
6. El sistema debe evitar guardar datos personales innecesarios.
7. El prototipo se diseñará para Fresnillo, pero podrá adaptarse a otros municipios.

6. Requisitos funcionales

RF-01. Gestión de proyectos

El sistema debe permitir crear, editar, consultar y archivar proyectos de estimación. Cada proyecto deberá incluir nombre, descripción, cliente, área solicitante, fecha, responsable, estado, tipo de sistema y prioridad.

RF-02. Captura de contexto municipal

El sistema debe capturar contexto del proyecto: área usuaria, proceso actual, problema administrativo, objetivo, restricción presupuestal, urgencia, dependencias internas, normativa aplicable y responsables de validación.

RF-03. Captura de módulos

El sistema debe permitir registrar módulos, pantallas, reportes, catálogos, roles, permisos, flujos e integraciones. Cada módulo tendrá complejidad funcional, nivel de claridad y nivel de criticidad.

RF-04. Historias de usuario y criterios de aceptación

El sistema debe permitir capturar historias de usuario con formato flexible: actor, necesidad, beneficio, reglas, datos requeridos, criterios de aceptación y evidencia esperada.

RF-05. Evaluación de madurez del requerimiento

El sistema debe calificar cada requerimiento en una escala de 1 a 5: incompleto, ambiguo, entendible, validado o listo para estimar. La calificación modificará el rango de incertidumbre.

RF-06. Modo de desarrollo

El sistema debe permitir seleccionar modo tradicional, asistido, bytecoding o híbrido. Cada modo tendrá factores ajustables para codificación, revisión, pruebas, documentación, refactorización y soporte.

RF-07. Estimación de esfuerzo técnico

El sistema debe calcular horas o jornadas por módulo y por etapa: análisis, diseño, codificación, revisión, pruebas, documentación, despliegue, capacitación y soporte inicial.

RF-08. Escenarios de esfuerzo

El sistema debe generar tres escenarios: optimista, probable y conservador. El escenario conservador aumentará cuando existan requerimientos ambiguos, integraciones externas, datos sensibles, cambios probables o poca disponibilidad del área usuaria.

RF-09. Equipo de trabajo

El sistema debe permitir capturar perfiles: líder técnico, desarrollador senior, desarrollador junior, analista, tester, diseñador, soporte, contador o administrativo. Cada perfil tendrá costo mensual, disponibilidad, modalidad de contratación y duración.

RF-10. Cálculo fiscal-laboral

El sistema debe calcular costo mensual del equipo considerando sueldo, prestaciones, carga social estimada, impuesto sobre nóminas, herramientas, licencias, administración y otros gastos. Las tasas deben ser editables.

RF-11. Cálculo de IVA e ISR preliminar

El sistema debe calcular subtotal, IVA trasladado, IVA acreditable estimado, total con IVA, gastos deducibles, gastos no deducibles, utilidad antes de ISR e ISR estimado. Debe advertir que es una simulación preliminar.

RF-12. Flujo de efectivo

El sistema debe capturar anticipos, pagos por entregables, pagos mensuales y pago final. Debe comparar ingresos esperados contra egresos mensuales y determinar capital de trabajo requerido.

RF-13. Control de cambios

El sistema debe registrar cambios con fecha, solicitante, descripción, causa, módulo afectado, clasificación, impacto en tiempo, impacto en costo, pruebas afectadas, capacitación afectada y decisión: aceptado, rechazado, pendiente o incluido.

RF-14. Mantenimiento y soporte

El sistema debe estimar mantenimiento mensual posterior, garantía, soporte correctivo, soporte evolutivo, hosting, respaldos, monitoreo, actualizaciones, documentación y capacitación.

RF-15. Reportes

El sistema debe generar tres reportes: reporte ejecutivo para Ayuntamiento, reporte de viabilidad para proveedor y reporte académico para investigación. Los reportes deben poder exportarse a PDF.

7. Requisitos no funcionales

RNF-01. Usabilidad

La interfaz debe ser clara para usuarios no técnicos. Debe explicar términos como IVA acreditable, capital de trabajo, modo bytecoding, cambio de alcance y costo patronal.

RNF-02. Mantenibilidad

El código debe separarse por capas: interfaz, API, dominio, motor de cálculo, persistencia y reportes.

RNF-03. Auditabilidad

Cada cálculo debe guardar parámetros usados, fecha, usuario y versión. No debe sobrescribir estimaciones anteriores sin bitácora.

RNF-04. Seguridad

El sistema debe implementar roles: administrador, estimador, consulta Ayuntamiento, proveedor y auditor. Debe proteger información financiera y evitar datos personales innecesarios.

RNF-05. Portabilidad

La primera versión debe poder ejecutarse localmente y luego migrar a servidor. Debe usar tecnologías comunes y documentadas.

RNF-06. Parametrización

Las tasas de IVA, ISR, UMA, salario mínimo, impuesto sobre nóminas, cargas estimadas y factores de modo de desarrollo deben modificarse sin cambiar código.

RNF-07. Calidad de reportes

Los reportes deben ser legibles, sin saturación de tablas, con conclusiones claras y advertencias de riesgo.

8. Modelo de datos conceptual

Proyecto

Campos principales: id, nombre, descripción, cliente, tipo de cliente, área, fecha, estado, prioridad, responsable, objetivo, presupuesto estimado, fecha objetivo y notas.

Módulo

Campos principales: id, proyecto_id, nombre, descripción, tipo, complejidad, claridad, criticidad, cantidad de pantallas, cantidad de reportes, integraciones, datos sensibles y observaciones.

Historia de usuario

Campos principales: id, módulo_id, actor, necesidad, beneficio, reglas, datos, criterios_aceptacion, prioridad, madurez, riesgos y estado.

Perfil de equipo

Campos principales: id, nombre, rol, nivel, salario_mensual, costo_hora, disponibilidad, modalidad, meses_asignados, factor_productividad y riesgo_salida.

Parámetro fiscal

Campos principales: id, año, estado, nombre, valor, unidad, fundamento, fecha_inicio, fecha_fin y notas.

Estimación

Campos principales: id, proyecto_id, modo_desarrollo, escenario, horas_analisis, horas_diseno, horas_codigo, horas_revision, horas_pruebas, horas_documentacion, horas_despliegue, horas_capacitacion, costo_directo, costo_indirecto, subtotal, iva, total, margen y riesgo.

Cambio

Campos principales: id, proyecto_id, módulo_id, fecha, solicitante, descripcion, tipo, impacto_tiempo, impacto_costo, impacto_pruebas, decision y observaciones.

Flujo de efectivo

Campos principales: id, proyecto_id, mes, ingresos, egresos_nomina, egresos_impuestos, egresos_herramientas, egresos_admin, saldo_mes, saldo_acumulado y capital_requerido.

9. Motor de estimación

Estimación técnica base

La estimación técnica base se calcula por módulo. Cada módulo recibe una puntuación según complejidad funcional, claridad, integraciones, datos, reportes, seguridad y disponibilidad del cliente.

Fórmula conceptual:

$$\text{esfuerzo_base} = \text{puntos_funcionales_internos} * \text{factor_complejidad} * \text{factor_claridad} * \text{factor_integraciones} * \text{factor_seguridad}$$

El sistema debe usar rangos, no un número único.

- Optimista: $\text{esfuerzo_base} * 0.85$
- Probable: $\text{esfuerzo_base} * 1.00$
- Conservador: $\text{esfuerzo_base} * 1.25$ a 1.80 según incertidumbre

Ajuste por modo de desarrollo

Cada modo de desarrollo modifica la distribución del esfuerzo.

Tradicional: mayor codificación manual, menor esfuerzo de prompting, revisión normal.

Asistido: menor codificación repetitiva, mayor revisión de sugerencias, apoyo en pruebas y documentación.

Bytecoding: menor tiempo de prototipo, mayor esfuerzo obligatorio de validación, arquitectura, refactorización y endurecimiento.

Híbrido: combina reducción moderada con revisión moderada.

Ejemplo conceptual:

$$\text{esfuerzo_total} = \text{codificacion} + \text{revision} + \text{pruebas} + \text{documentacion} + \text{despliegue} + \text{soporte}$$

En bytecoding, codificación puede bajar, pero revisión y pruebas suben si la criticidad o la complejidad aumentan.

Cálculo de cambios

Cada cambio tendrá impacto:

$$\text{impacto_cambio} = \text{impacto_modulo} + \text{impacto_datos} + \text{impacto_pruebas} + \text{impacto_capacitacion} + \text{impacto_documentacion}$$

Clasificación:

- Corrección: error contra alcance aceptado.
- Garantía: falla dentro de lo comprometido.
- Ajuste menor: cambio pequeño sin alterar modelo de datos.
- Mejora: nuevo comportamiento no incluido.
- Nuevo alcance: módulo, integración o regla nueva.

Costo de equipo

El costo de equipo debe calcularse por perfil y mes.

$$\text{costo_perfil_mensual} = \text{salario_mensual} + \text{prestaciones} + \text{cuotas_seguridad_social} + \text{impuesto_sobre_nominas} + \text{herramientas} + \text{costo_administrativo}$$

El sistema debe permitir dos formas:

1. Cálculo detallado por parámetros.
2. Factor de carga patronal estimado cuando no se tenga detalle.

IVA e ISR preliminar

$$\text{subtotal} = \text{costo_total} + \text{margen_objetivo}$$

$$\text{IVA trasladado} = \text{subtotal} * \text{tasa_IVA}$$

$$\text{total_facturado} = \text{subtotal} + \text{IVA trasladado}$$

$$\text{utilidad_antes_ISR} = \text{subtotal} - \text{gastos_deducibles} - \text{gastos_no_deducibles_ajustados}$$

$$\text{ISR estimado} = \text{utilidad_fiscal_estimada} * \text{tasa_ISR}$$

El sistema debe mostrar advertencia: el cálculo es preliminar y requiere revisión profesional.

Flujo de efectivo

$$\text{saldo_mes} = \text{ingresos_mes} - \text{egresos_mes}$$
$$\text{saldo_acumulado} = \text{saldo_acumulado_anterior} + \text{saldo_mes}$$
$$\text{capital_trabajo_requerido} = \text{valor absoluto del saldo acumulado mínimo si es negativo}$$

Si capital_trabajo_requerido supera la capacidad declarada del proveedor, el sistema debe marcar riesgo alto.

10. Arquitectura propuesta

Arquitectura general

Se recomienda una arquitectura web modular.

- Frontend: React, Vue o Next.js.
- Backend: Node.js con Express o Python con FastAPI.
- Base de datos: PostgreSQL o SQLite para prototipo local.
- Motor de cálculo: módulo de dominio separado.
- Reportes: generación HTML a PDF.
- Exportaciones: CSV y XLSX.
- Autenticación: roles básicos.

Capas

Capa de presentación. Formularios, panel de proyectos, captura de módulos, comparador de escenarios, flujo de efectivo y reportes.

Capa de aplicación. Casos de uso: crear proyecto, estimar, registrar cambio, calcular flujo, generar reporte.

Capa de dominio. Entidades, reglas de estimación, reglas fiscales parametrizadas, reglas de riesgo.

Capa de infraestructura. Base de datos, repositorios, exportaciones, generación PDF.

Capa de configuración. Parámetros por año, estado, modo de desarrollo y factores de riesgo.

11. API sugerida

Proyectos

- GET /api/projects
- POST /api/projects
- GET /api/projects/{id}
- PUT /api/projects/{id}
- DELETE /api/projects/{id}

Módulos e historias

- POST /api/projects/{id}/modules
- PUT /api/modules/{id}
- POST /api/modules/{id}/stories
- PUT /api/stories/{id}

Estimaciones

- POST /api/projects/{id}/estimate
- GET /api/projects/{id}/estimates
- GET /api/estimates/{id}

Parámetros

- GET /api/parameters
- POST /api/parameters
- PUT /api/parameters/{id}

Cambios

- POST /api/projects/{id}/changes
- GET /api/projects/{id}/changes
- PUT /api/changes/{id}

Reportes

- GET /api/projects/{id}/report/municipal
- GET /api/projects/{id}/report/provider
- GET /api/projects/{id}/report/research

12. Flujos de interfaz

Flujo 1. Crear proyecto

1. Capturar datos generales.
2. Seleccionar tipo de sistema.

3. Capturar área solicitante.
4. Definir objetivo administrativo.
5. Guardar proyecto en estado borrador.

Flujo 2. Capturar requerimientos

1. Crear módulo.
2. Crear historias de usuario.
3. Capturar criterios de aceptación.
4. Evaluar claridad.
5. Marcar riesgos.

Flujo 3. Estimar escenarios

1. Seleccionar modo de desarrollo.
2. Seleccionar equipo.
3. Calcular esfuerzo.
4. Calcular costo.
5. Comparar escenarios.
6. Guardar versión de estimación.

Flujo 4. Analizar viabilidad fiscal-laboral

1. Capturar perfiles y salarios.
2. Capturar modalidad de contratación.
3. Aplicar parámetros 2026.
4. Calcular costo de equipo.
5. Calcular subtotal, IVA, ISR y margen.
6. Marcar riesgo financiero.

Flujo 5. Flujo de efectivo

1. Capturar anticipo.
2. Capturar pagos por entregable.
3. Capturar pago final.
4. Comparar egresos mensuales.
5. Calcular capital de trabajo requerido.

Flujo 6. Control de cambios

1. Registrar solicitud de cambio.
2. Clasificar tipo de cambio.
3. Estimar impacto.
4. Decidir si se acepta, rechaza o cotiza.
5. Actualizar reporte.

13. Reportes

Reporte para Ayuntamiento

Debe incluir:

- Resumen ejecutivo.
- Alcance estimado.
- Riesgos técnicos.
- Riesgos de cambios.
- Mantenimiento requerido.
- Comparación de modos de desarrollo.
- Señales de alerta de cotización baja.
- Recomendación de pagos por entregables.
- Checklist de aceptación.

Reporte para proveedor

Debe incluir:

- Costo técnico.
- Costo fiscal-laboral.
- Flujo de efectivo.
- Capital de trabajo necesario.
- Margen estimado.
- Riesgo de personal.
- Riesgo de cambios.
- Precio mínimo sostenible.
- Recomendación de anticipo y mantenimiento.

Reporte académico

Debe incluir:

- Variables capturadas.
- Escenarios comparados.
- Resultados del caso.
- Diferencias entre modos de desarrollo.
- Evidencia para validar o ajustar hipótesis.

14. Backlog inicial

Épica 1. Gestión de proyectos

Historia: Como estimador, quiero crear un proyecto para organizar la información del cliente, alcance y contexto.

Criterio de aceptación: El proyecto queda guardado y puede consultarse con estado borrador.

Épica 2. Captura de requerimientos

Historia: Como analista, quiero registrar módulos e historias para estimar con base en información temprana.

Criterio de aceptación: Cada historia tiene actor, necesidad, beneficio y criterios de aceptación.

Épica 3. Comparación de modos de desarrollo

Historia: Como responsable técnico, quiero comparar tradicional, asistido y bytecoding para ver diferencias de esfuerzo y riesgo.

Criterio de aceptación: El sistema muestra tres escenarios con rangos y explicación.

Épica 4. Cálculo fiscal-laboral

Historia: Como proveedor, quiero calcular costo real del equipo para evitar cotizar por debajo de mi costo.

Criterio de aceptación: El sistema muestra costo mensual, impuestos estimados, margen y alertas.

Épica 5. Flujo de efectivo

Historia: Como proveedor, quiero saber cuánto capital de trabajo necesito antes de cobrar.

Criterio de aceptación: El sistema muestra saldo mensual y capital requerido.

Épica 6. Control de cambios

Historia: Como Ayuntamiento, quiero registrar cambios para distinguir garantía, mejora y nuevo alcance.

Criterio de aceptación: Cada cambio tiene impacto y decisión.

Épica 7. Reportes

Historia: Como usuario, quiero generar PDF para presentar la estimación.

Criterio de aceptación: El reporte incluye resumen, escenarios, riesgos y recomendaciones.

15. Criterios de aceptación globales

1. El sistema permite capturar un proyecto completo.
2. El sistema calcula al menos tres escenarios de esfuerzo.
3. El sistema compara modos de desarrollo.
4. El sistema calcula costo fiscal-laboral con parámetros editables.
5. El sistema calcula flujo de efectivo.
6. El sistema registra cambios.
7. El sistema exporta reportes.
8. El sistema guarda versiones de estimación.
9. El sistema muestra advertencias cuando la estimación sea incompleta.
10. El sistema puede usarse sin conexión en modo prototipo local.

16. Plan de pruebas

Pruebas funcionales

- Crear proyecto.
- Capturar módulos.
- Capturar historias.
- Calcular estimación.
- Registrar cambios.
- Generar reportes.

Pruebas de cálculo

- Validar IVA con tasa configurable.
- Validar ISR preliminar con tasa configurable.
- Validar impuesto sobre nóminas con tasa configurable.
- Validar capital de trabajo.
- Validar cambios de escenario.

Pruebas de usabilidad

- Usuario administrativo crea proyecto sin ayuda técnica.
- Usuario técnico captura módulos.
- Usuario proveedor interpreta reporte de viabilidad.

Pruebas de regresión

- Modificar parámetro fiscal y recalculan sin romper proyectos anteriores.

- Guardar estimación nueva sin borrar versión anterior.

17. Riesgos del proyecto

1. Falta de datos locales reales.
2. Cambios fiscales o laborales posteriores a 2026.
3. Sobreconfianza en herramientas generativas.
4. Captura incompleta por parte del usuario.
5. Confusión entre precio bajo y precio viable.
6. Resistencia del proveedor a transparentar costos.
7. Falta de mantenimiento posterior.
8. Dificultad para validar con casos reales.

18. Entregables Markdown para implementación

El paquete de implementación debe incluir los siguientes archivos:

1. README.md
2. 00_contexto_investigacion.md
3. 01_modelo_integrado.md
4. 02_variables_estimacion.md
5. 03_parametros_fiscales_laborales_2026.md
6. 04_srs_requisitos.md
7. 05_sds_diseno.md
8. 06_modelo_datos.md
9. 07_motor_formulas.md
10. 08_flujos_ui.md
11. 09_api_contract.md
12. 10_backlog_historias_usuario.md
13. 11_criterios_aceptacion.md
14. 12_plan_pruebas.md
15. 13_reportes.md
16. 14_prompt_para_claude.md
17. 15_seguridad_privacidad.md
18. 16_validacion_investigacion.md
19. 17_seed_data_parametros_2026.json
20. 18_checklist_ayuntamiento_proveedor.md

19. Conclusión técnica

EMPS-Fresnillo debe ser más que una calculadora de horas. Su valor está en integrar variables que normalmente se separan: requerimientos, modo de desarrollo, bytecoding, cambios, fiscalidad, nómina, flujo de efectivo y mantenimiento. Si el prototipo logra mostrar cuándo una cotización es técnicamente posible pero financieramente débil, o cuándo un proyecto rápido por

prompts necesita más revisión y soporte, la investigación tendrá una aplicación práctica para el Ayuntamiento, proveedores y otros municipios.